

COMPE271 Project - Final Report

The C program and MIPS Assembly code submitted is my own original work.

COMPE-271

Brandon Nguyen, 827813045

Brandon Nguyen

Program Description

The goal of my project is to create a game that combines the elements of Connect 4 and a Dice Roll or random number generator feature, followed by other unique gameplay mechanics that make my game different from the standard Connect 4 game. When it comes to the turn base portion and the implementation of the dice roll/random number generator, odd numbers are assigned to Player 1 and even numbers are assigned to Player 2. The game automatically begins with a dice roll that will continue after each turn or placement of a piece (user input) and if the dice rolls or generates a number based on the set of numbers assigned to the respective players, then said player gets a turn to place a piece on the gameboard. Example: An odd number like 1, 3, or 5 means that Player 1 gets to place a piece on the board. Since the dice roll is randomly generated, the same type of numbers can be rolled consecutively, resulting in an advantage for the lucky player and potentially a very quick and exciting game. Additionally, I hope to later add other suggested and unique gameplay mechanics like a “pop-out” or blocker feature to remove the other player’s piece or block a space from having a piece placed there to extend the game.

Core Functions

The core functions that my project includes consist of a random number generator, conditional branches, arrays, and a 2D array. The first important core function that is seen in my project is the random number generator (Syscall 42), symbolizing a dice roll. The random number generator/dice roll is important because it randomly generates an integer 1-6 at the start of the game and each turn after that to dictate which player gets to place a piece on the gameboard. Therefore, the random number generator initiates the turn base aspect of the game and continuously plays a role each turn after a piece is put onto the board. The array and 2D array is also an essential part of the MIPS project program because the array stores the information or memory that equates to an 8x8 piece game board for a Connect 4 game. In relation to the array, a 2D array is then able to be produced in the output, allowing for easier visualization of the game to be played. If-else statements also play a large role because they are incorporated into all phases and aspects of the program. If-else statements can be found in the game loops (both creation and printing of the board), dice roll logic, the user input section, piece placement logic, and win and draw conditions. Overall, all of these important core functions come together to ensure that my MIPS Assembly project operates correctly. A unique feature that I was also able to implement was sound, which consisted of loading pitch, duration, instrument type, and volume into Syscall 31 to output a MIDI sound noise within MARS/MIPS Assembly.

C Program Version

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ROWS 8
#define COLS 8

void createBoard(char gameBoard[ROWS][COLS]);
void printBoard(char gameBoard[ROWS][COLS]);
int displayPiece(char gameBoard[ROWS][COLS], int column, char
player);
int winCondition(char gameBoard[ROWS][COLS], char player);
int drawCondition(char gameBoard[ROWS][COLS]);
int rollDice();

int main() {
    char gameBoard[ROWS][COLS];
    int column, piecePlace, winner, diceRoll;
    char turnPlayer;

    srand(time(0));

    createBoard(gameBoard);

    printf("Current Project C Program Draft 1\n");
    printBoard(gameBoard);

    while (1) {
```

```
    diceRoll = rollDice();
    if (diceRoll % 2 == 1) {
        turnPlayer = '1';
    } else {
        turnPlayer = '2';
    }

    printf("Dice rolled: %d. It's Player %c's turn.\n",
diceRoll, turnPlayer);

    printf("Player %c, enter a column number to display your
piece: ", turnPlayer, COLS);
    scanf("%d", &column);

    if (column < 1 || column > COLS) {
        printf("Error, try a different column number\n");
        continue;
    }

    piecePlace = displayPiece(gameBoard, column - 1,
turnPlayer);
    if (piecePlace == 0) {
        printf("Column is full. Please try a different
column number.\n");
        continue;
    }

    printBoard(gameBoard);

    winner = winCondition(gameBoard, turnPlayer);
```

```
        if (winner) {
            printf("Player %c is the winner.\n", turnPlayer);
            break;
        }

        if (drawCondition(gameBoard)) {
            printf("Game is a Draw.\n");
            break;
        }

        if (turnPlayer == '1') {
            turnPlayer = '2';
        }
        else {
            turnPlayer = '1';
        }
    }

    return 0;
}

void createBoard(char gameBoard[ROWS][COLS]) {
    int i, j;
    for (i = 0; i < ROWS; i++) {
        for (j = 0; j < COLS; j++) {
            gameBoard[i][j] = ' ';
        }
    }
}

void printBoard(char gameBoard[ROWS][COLS]) {
    int i, j;
    for (i = 0; i < ROWS; i++) {
```

```

        for ( j = 0; j < COLS; j++) {
            if (gameBoard[i][j] == ' ') {
                printf(". ");
            } else {
                printf("%c ", gameBoard[i][j]);
            }
        }
        printf("\n");
    }

    for (j = 0; j < COLS; j++) {
        printf("--");
    }
    printf("\n");

    for (j = 1; j <= COLS; j++) {
        printf("%d ", j);
    }

    printf("\n\n");
}

int displayPiece(char gameBoard[ROWS][COLS], int column, char
player) {
    for (int i = ROWS - 1; i >= 0; i--) {
        if (gameBoard[i][column] == ' ') {
            gameBoard[i][column] = player;
            return 1;
        }
    }
    return 0;
}

```

```

int winCondition(char gameBoard[ROWS][COLS], char player) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (j <= COLS - 4 && gameBoard[i][j] == player &&
gameBoard[i][j + 1] == player &&
                gameBoard[i][j + 2] == player && gameBoard[i][j
+ 3] == player)
                return 1;
            if (i <= ROWS - 4 && gameBoard[i][j] == player &&
gameBoard[i + 1][j] == player &&
                gameBoard[i + 2][j] == player && gameBoard[i +
3][j] == player)
                return 1;
            if (i <= ROWS - 4 && j <= COLS - 4 &&
gameBoard[i][j] == player &&
                gameBoard[i + 1][j + 1] == player && gameBoard[i
+ 2][j + 2] == player &&
                gameBoard[i + 3][j + 3] == player)
                return 1;
            if (i >= 3 && j <= COLS - 4 && gameBoard[i][j] ==
player &&
                gameBoard[i - 1][j + 1] == player && gameBoard[i
- 2][j + 2] == player &&
                gameBoard[i - 3][j + 3] == player)
                return 1;
        }
    }
    return 0;
}

int drawCondition(char gameBoard[ROWS][COLS]) {
    for (int j = 0; j < COLS; j++) {
        if (gameBoard[0][j] == ' ')
            return 0;
    }
}

```



```
        return 1;
    }

int rollDice() {
    return rand() % 6 + 1;
}
```

Tests

The implementation of my project for both the C program and MIPS assembly versions can be broken down into 3 main phases. These 3 major phases include the creation of the game board, the piece placement feature, and the random number generator/dice roll feature. Another major and challenging feature was the win and draw conditions and logic. Below will be provided a series of screenshots depicting the input of each phase and the progression of the project over time in both C and MIPS Assembly.

Creation of the game board (Array and 2D Array)

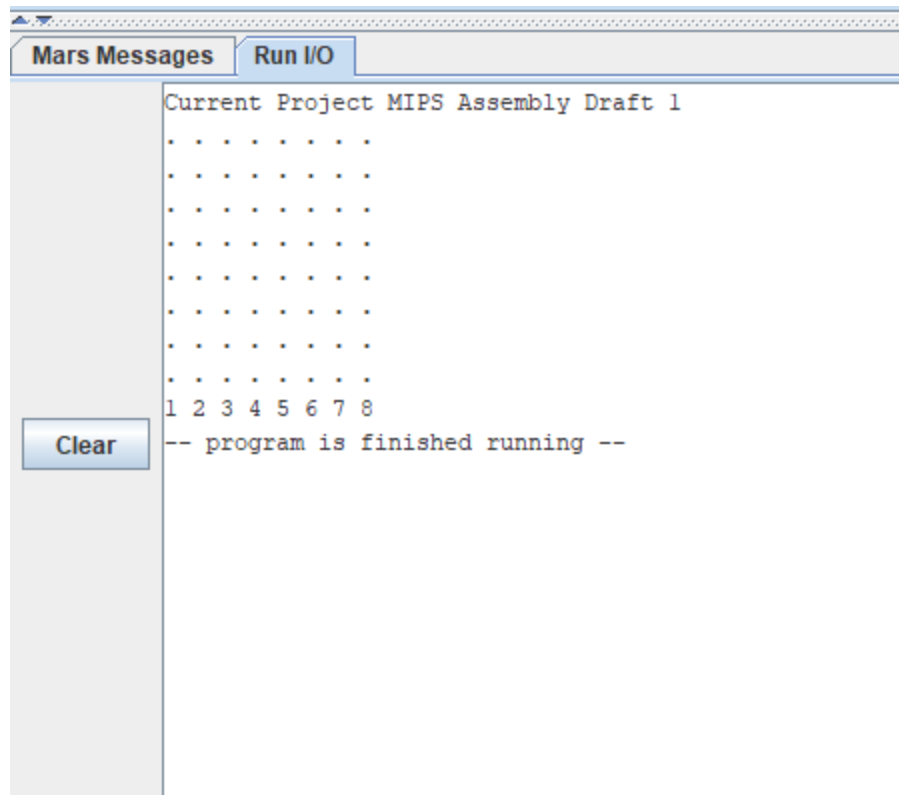
```

Player 1, enter a column number to display your piece: 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 1 . . .
1 2 . 2 . . .
1 2 . 1 . . .
-----
1 2 3 4 5 6 7 8

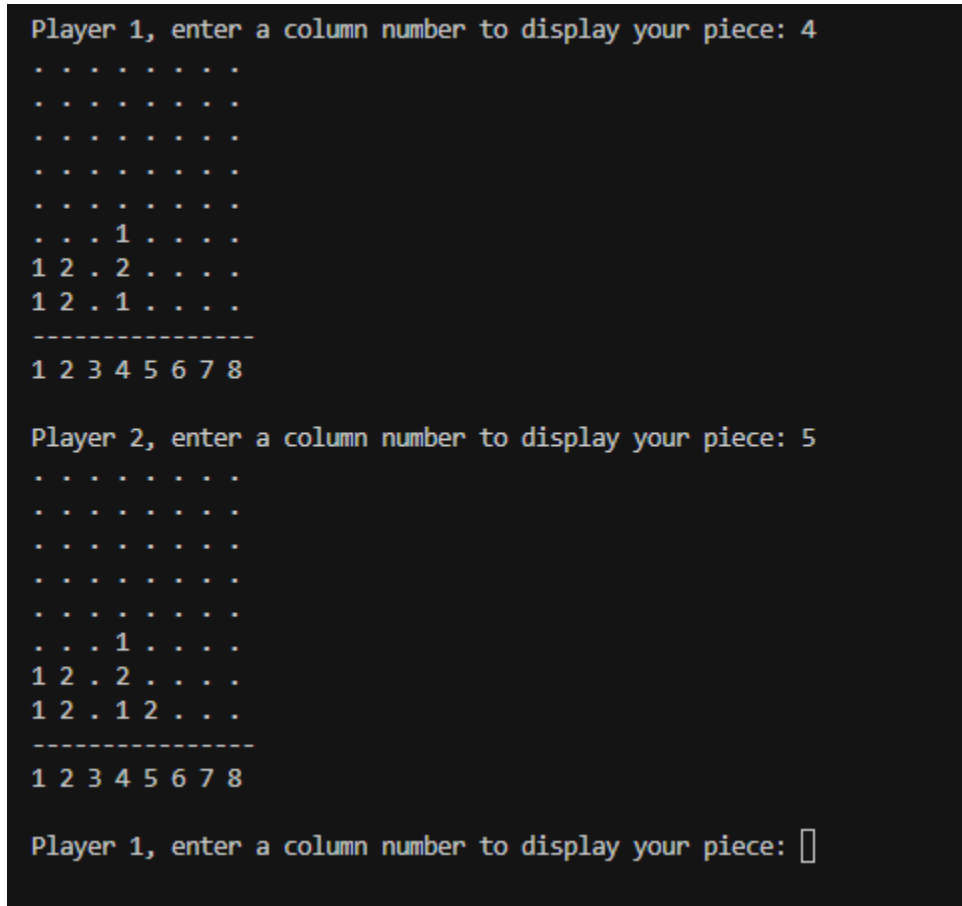
Player 2, enter a column number to display your piece: 5
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 1 . . .
1 2 . 2 . . .
1 2 . 1 2 . . .
-----
1 2 3 4 5 6 7 8

Player 1, enter a column number to display your piece: 

```



Piece placement (User Input)



Mars Messages	Run I/O
Clear	Player %c, enter a column number to display your piece: 7 . X X . X . X . X . 1 2 3 4 5 6 7 8
	Player %c, enter a column number to display your piece: 8 . X X . X . X . X X 1 2 3 4 5 6 7 8
	Player %c, enter a column number to display your piece:

Mars Messages	Run I/O
Clear	Player 1, enter a column number to display your piece: 4 . 1 . 1 . . . 1 2 3 4 5 6 7 8
	Player 1, enter a column number to display your piece: 4 . 1 1 . 1 . . . 1 2 3 4 5 6 7 8
	Player 1, enter a column number to display your piece:

Dice Roll (Random number generator)

```

Dice rolled: 3. It's Player 1's turn.
Player 1, enter a column number to display your piece: 1
1 . . . . .
2 . . . . .
1 . . . . .
1 . . . . .
2 . . . . .
1 . . . . .
2 . . . . .
1 2 . . . . .
-----
1 2 3 4 5 6 7 8

Dice rolled: 4. It's Player 2's turn.
Player 2, enter a column number to display your piece: 9
Error, try a different column number
Dice rolled: 2. It's Player 2's turn.
Player 2, enter a column number to display your piece: 

```

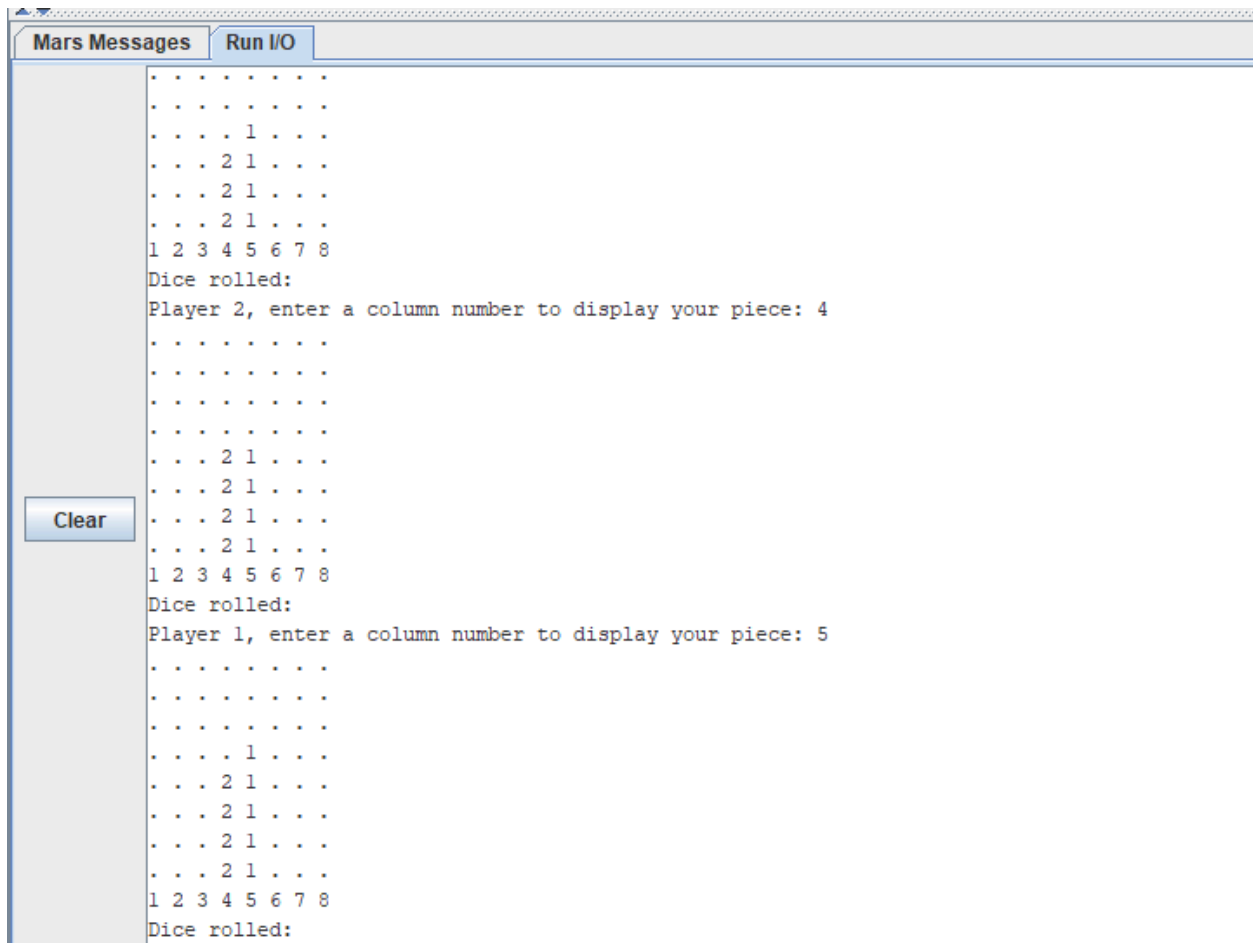
```

Dice rolled: 2. It's Player 2's turn.
Player 2, enter a column number to display your piece: 5
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 2 . . .
. . . 1 2 . . .
. . . 1 2 . . .
-----
1 2 3 4 5 6 7 8

Dice rolled: 6. It's Player 2's turn.
Player 2, enter a column number to display your piece: 5
. . . . .
. . . . .
. . . . .
. . . . .
. . . 2 . . .
. . . 2 . . .
. . . 1 2 . . .
. . . 1 2 . . .
-----
1 2 3 4 5 6 7 8

Player 2 is the winner.
PS C:\Users\brand\OneDrive\Desktop\C Programming> 

```



Win and draw conditions (Logic)

```

Player 1, enter a column number to display your piece: 4
. . . . .
. . . . .
. . . . .
. . . . .
. . 1 . . .
. . 1 2 . . .
. 1 2 1 1 . .
1 2 2 1 2 . . 2
-----
1 2 3 4 5 6 7 8

Player 1 is the winner.
PS C:\Users\brand\OneDrive\Desktop\C Programming>

```

```
Player 2, enter a column number to display your piece: 8
1 1 1 2 1 1 2 2
2 2 2 1 2 2 1 1
1 1 1 2 1 1 2 2
2 2 2 1 2 2 1 1
1 1 1 2 1 1 2 2
2 2 2 1 2 2 1 1
1 1 1 2 1 1 2 2
2 2 2 1 2 2 1 1
1 1 1 2 1 1 2 2
1 2 2 2 1 2 1 2
-----
1 2 3 4 5 6 7 8

Game is a Draw.
PS C:\Users\brand\OneDrive\Desktop\C Programming> █
```

```
Player 2, enter a column number to disp
. . . . .
. . . . .
. . . . .
. . . . .
. . . 2 . . .
. . . 2 . . .
. . 1 2 . . .
. . 1 2 . . .
1 2 3 4 5 6 7 8
Player 2 is the winner.
-- program is finished running --
```

```
Player 2, enter a column number to display your piece: 6
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . 2 2 . . . 1
. . 2 2 2 2 . 1
1 2 3 4 5 6 7 8
Player 2 is the winner.
-- program is finished running --
```

```

Player 2, enter a column number to display your piece: 5
. . . . .
. . . . .
. . . . .
. . . . .
. . . . 2 .
. . . . 2 1 . 1
1 . . 2 1 1 . 1
1 . 2 1 2 1 . 1
1 2 3 4 5 6 7 8
Player 2 is the winner.
-- program is finished running --

```

```

Dice rolled: 5
Player 1, enter a column number to display your piece: 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . 1 . . .
. . . 1 1 . .
. . . 2 2 1 .
. . . 2 2 2 1 .
1 2 3 4 5 6 7 8
Player 1 is the winner.
-- program is finished running --

```

Clear

```

Dice rolled: 5
Player 1, enter a column number to display your piece: 8
2 1 2 1 1 2 1 1
1 1 1 2 1 2 2 2
1 1 1 2 1 1 2 1
1 2 2 1 2 1 1 2
2 2 2 1 2 2 1 2
2 2 1 2 2 2 2 1
2 1 1 1 2 1 1 2
2 2 1 2 2 2 2 1
1 2 3 4 5 6 7 8
Game is a Draw.
-- program is finished running --

```


Progress Summary

This project and class consisted of creating a video game project in C programming and MIPS Assembly language. We started with outlining our project with ideas and learning about important functions that would be used for our games. Building a draft version of our project using the C program was the next crucial step. Both the project outline and the C program draft took a good amount of time. The rest of the semester revolved around the MIPS Assembly code, and translating over the C program version of our game onto MIPS Assembly. This part also took a long time as many of us were learning MIPS along the way and I specifically spent most of my time translating the core/basic version of my game, not having much time for any unique features. In MIPS Assembly, creating and printing the arrays of the board was the easiest part, followed by implementing a random number generator after learning the specific syscall function. On the other hand, the turn switching and win and draw condition logic took the longest for me to implement. Sound was the only additional feature added to the code. In the end, I was able to produce a fully functioning Connect 4 game in MIPS Assembly with all the necessary passing requirements and one unique feature or two like the dice roll.

Uncompleted Tasks

Unfortunately, I was unable to add any major and unique game features that would have differentiated my project from the standard Connect 4 game. A couple unique gameplay mechanics that were suggested to me were the blocker feature and the popout feature. Both the blocker and the popout feature would have most likely allowed players to do something strategically to their opponent by filling in a specific spot or removing a game piece from the board when the board senses that a player has 3 pieces in a row. I feel like both of these features would be incorporated somewhere in the win and draw condition logic, which took me a good amount of time to implement and get working. Without these unique game features, my game hardly differentiates from the traditional Connect 4 game outside of the dice roll feature.

Conclusion

If I had to do it all over again, things I would do differently consist of adding the unique features that I was unable to get to, and devoting more time and effort towards the MIPS Assembly coding. With more time, I would have been able to go onto more interesting things like unique game features or a bitwise display. The MIPS Assembly portion of the code or project is the most important thing so I would have started even earlier than I did and most likely would have been able to add more unique features.

Estimated Work Time

> 160 hours

MIPS Assembly Version (Final Project)

```
# Author: Brandon Nguyen, 827813045
# Final Project - Connect 4 w/Turn Based Dice Roll
# COMPE 271
```

```
.data
ROWS: .word 8
COLS: .word 8
gameBoard: .space 64
projectTitle: .ascii "MIPS Assembly Project - Connect 4\n"
diceRoll: .ascii "Dice rolled: "
playerOneTurn: .ascii "\nPlayer 1, enter a column number to display your piece: "
playerTwoTurn: .ascii "\nPlayer 2, enter a column number to display your piece: "
invalidColumn: .ascii "Error, try a different column number "
fullColumn: .ascii "Column is full. Please try a different column number: "
winCondition1: .ascii "Player 1 is the winner."
winCondition2: .ascii "Player 2 is the winner."
drawCondition: .ascii "Game is a Draw."
```

```
.text
```

#PART 1 - MAIN LOOPS

MAIN

Initialize variables

```
li    $t0, 0          # Initialize $t0 (row counter) to 0.
li    $t1, 0          # Initialize $t1 (column counter) to 0.
la    $a0, gameBoard  # Load the address of the game board into $a0.
li    $t7, 1          # Initialize $t7 to 1, representing Player 1's turn.
```

Create the board

createBoard:

```
lw    $t2, ROWS       # Load the number of rows into $t2.
lw    $t3, COLS       # Load the number of columns into $t3.
```

outerLoop:

```
beq    $t0, $t2, printTitle # If all rows are processed, jump to printTitle.
li    $t1, 0          # Reset column counter $t1 to 0.
```

innerLoop:

 beq \$t1, \$t3, createBoardRows # If all columns in the current row are processed,
jump to createBoardRows.

```

    li    $t4, ' '           # Load a space character into $t4.
    sb    $t4, 0($a0)        # Store the space character at the current address of $a0.
    addi   $a0, $a0, 1        # Move to the next memory address in the game board.
    addi   $t1, $t1, 1        # Increment column counter $t1.
    j      innerLoop

```

createBoardRows:

```

    addi   $t0, $t0, 1        # Increment row counter $t0.
    j      outerLoop

```

printTitle:

```

    li    $v0, 4
    la    $a0, projectTitle
    syscall

```

Print the board

```

    la    $a0, gameBoard     # Reload the address of the game board into $a0.
    jal   printBoard

```

Print board function

printBoard:

```

    li    $t0, 0             # Initialize row counter $t0 to 0.
    lw    $t2, ROWS          # Load the number of rows into $t2.
    lw    $t3, COLS          # Load the number of columns into $t3.
    la    $a1, gameBoard     # Load the address of the game board into $a1.

```

outerLoopPrint:

```

    beq   $t0, $t2, numbersPrint # If all rows are printed, jump to numbersPrint.
    li    $t1, 0              # Reset column counter $t1 to 0.

```

innerLoopPrint:

 beq \$t1, \$t3, newlinePrint # If all columns in the current row are printed, jump to
newlinePrint.

```

    lb    $t4, 0($a1)        # Load the current cell's value into $t4.
    beq   $t4, ' ', dotPrint  # If the cell is empty (space), jump to dotPrint.
    li    $v0, 11             # Set syscall for printing a character.

```

```

    move    $a0, $t4        # Move the cell's value to $a0.
    syscall                                # Print the cell's value.
    j       spacePrint

```

dotPrint:

```

    li      $v0, 11          # Set syscall for printing a character.
    li      $a0, '.'         # Load '.' into $a0.
    syscall                                # Print '.'.

```

spacePrint:

```

    li      $v0, 11          # Set syscall for printing a character.
    li      $a0, ' '        # Load a space into $a0.
    syscall                                # Print the space.
    addi    $a1, $a1, 1      # Move to the next cell in the game board.
    addi    $t1, $t1, 1      # Increment column counter $t1.
    j       innerLoopPrint

```

newlinePrint:

```

    li      $v0, 11          # Set syscall for printing a character.
    li      $a0, 10          # Load newline character into $a0.
    syscall                                # Print newline.
    addi    $t0, $t0, 1      # Increment row counter $t0.
    j       outerLoopPrint

```

Print the row of column numbers

numbersPrint:

```

    li      $t0, 1           # Initialize column number to 1.
    lw      $t2, COLS        # Load the number of columns into $t2.

```

ColPrint:

```

    ble     $t0, $t2, printNumber # If $t0 is less than or equal to the column count, jump
to printNumber.

```

printNumber:

```

    li      $v0, 1           # Set syscall for printing an integer.
    move    $a0, $t0         # Move the column number to $a0.
    syscall                                # Print the column number.
    li      $v0, 11          # Set syscall for printing a character.
    li      $a0, ' '        # Load a space into $a0.
    syscall                                # Print the space.

```

```
    addi    $t0, $t0, 1        # Increment the column number.
    ble     $t0, $t2, ColPrint # Repeat if there are more columns to print.

# Print a newline
    li      $v0, 11
    li      $a0, 10
    syscall

# Main logic: Call functions for board checks
    jal     VerticalBoardCheck
    jal     HorizontalBoardCheck
    jal     DiagonalCheckBackward
    jal     DiagonalCheckForward
    jal     checkDraw
# Parent (Main Loops) to Children

# DICE ROLL FUNCTION
printdiceRoll: #Parent

    li      $v0, 4
    la      $a0, diceRoll
    syscall

# Generate a random number from 1 to 6
    li      $a1, 6             # Set the maximum random number (6)
    li      $v0, 42            # RANDOM NUMBER GENERATOR FUNCTION
    syscall
    addi    $a0, $a0, 1

# Print the random number
    li      $v0, 1
    move    $a0, $a0
    syscall

# Check if the random number is odd or even - turn based dependent feature
    andi    $t0, $a0, 1        # $t0 = random number AND 1
    beq     $t0, 0, setPlayerTwoTurn # If even, go to Player 2's turn
    j       setPlayerOneTurn    # If odd, go to Player 1's turn
```

setPlayerOneTurn:

```
li    $t7, 1          # Set $t7 to 1 (Player 1's turn)
j     inputAndTurn
```

setPlayerTwoTurn:

```
li    $t7, 2          # Set $t7 to 2 (Player 2's turn)
j     inputAndTurn
```

PART 2 - PIECE PLACEMENT

inputAndTurn: # Checks which player's turn it is

```
li    $v0, 4          # Set syscall for printing a string.
beq   $t7, 1, p1turn  # If $t7 is 1 (Player 1's turn), jump to p1turn.
j     p2turn          # Otherwise, jump to p2turn.
```

p1turn:

```
la    $a0, playerOneTurn # Load the address of the Player 1's turn message.
syscall                # Print the Player 1's turn message.
j     mainLoop
```

p2turn:

```
la    $a0, playerTwoTurn # Load the address of the Player 2's turn message.
syscall                # Print the Player 2's turn message.
j     mainLoop
```

Reads Column Number Input

mainLoop:

```
li    $v0, 5          # Set syscall for reading an integer input.
syscall                # Read user input and store it in $v0.
move   $t0, $v0        # Move the input value to $t0 for processing.
```

```
li    $t1, 1          # Load the minimum column number (1) into $t1.
bge   $t0, $t1, maxColCheck # If the input is >= 1, jump to maxColCheck.
j     invalidColumnMessage
```

maxColCheck:

```
lw    $t1, COLS        # Load the maximum column number into $t1.
ble   $t0, $t1, placePiece # If the input is <= max column, jump to placePiece.
j     invalidColumnMessage
```

Invalid Column Logic Check

invalidColumnMessage:

```
li    $v0, 4          # Set syscall for printing a string.
la    $a0, invalidColumn # Load the address of the invalid column message.
syscall                # Print the invalid column message.
j     inputAndTurn     # Restart the turn.
```

Placing a Piece

placePiece:

```
addi   $t0, $t0, -1    # Convert the column number to a zero-based index.
la     $t1, gameBoard   # Load the address of the game board into $t1.
li     $t2, 7           # Set the row index to the bottom row (7).
```

placePieceLoop:

```
mul     $t3, $t2, 8     # Calculate the row's base address (row index * 8 columns).
add     $t3, $t3, $t0    # Add the column offset to get the cell's address.
add     $t3, $t1, $t3    # Add the base game board address to get the absolute
address.
```

```
lb     $t4, 0($t3)      # Load the value at the calculated cell address.
```

```
li     $t5, ' '         # Load a space character into $t5.
```

```
bne    $t4, $t5, placePieceNextRow # If the cell is not empty, jump to
placePieceNextRow.
```

```
beq    $t7, 1, PlayerOnePlace # If it's Player 1's turn, jump to PlayerOnePlace.
```

```
j     PlayerTwoPlace    # Otherwise, jump to PlayerTwoPlace.
```

PlayerOnePlace: # Player 1 places a piece

```
li     $t6, '1'         # Load '1' (Player 1's piece) into $t6.
sb     $t6, 0($t3)      # Store the piece at the calculated cell address.
```

SOUND 1 - PLAYER 1 PIECE PLACEMENT SOUND

```
li $v0, 31 # Sound
la $t0, 60 # Pitch
la $t1, 100 # Duration
la $t2, 4 # Instrumnt Type
la $t3, 127 # Volume
move $a0, $t0
move $a1, $t1
```

```
move $a2, $t2
move $a3, $t3
syscall
```

```
li    $t7, 2          # Set the turn to Player 2.
j     printAndLoop
```

PlayerTwoPlace: # Player 2 places a piece

```
li    $t6, '2'         # Load '2' (Player 2's piece) into $t6.
sb    $t6, 0($t3)      # Store the piece at the calculated cell address.
```

SOUND 2 - PLAYER 2 PIECE PLACEMENT SOUND

```
li $v0, 31 # Sound
la $t0, 69 # Pitch
la $t1, 100 # Duration
la $t2, 58 #Instrument Type
la $t3, 127 #Volume
move $a0, $t0
move $a1, $t1
move $a2, $t2
move $a3, $t3
syscall
```

```
li    $t7, 1          # Set the turn to Player 1.
j     printAndLoop
```

placePieceNextRow:

```
addi  $t2, $t2, -1     # Move to the row above.
bgez  $t2, placePieceLoop # If the row index is >= 0, repeat the loop.
j     fullColumnMessage # If all rows are full, jump to fullColumnMessage.
```

Full Column Logic Check

fullColumnMessage:

```
li    $v0, 4
la    $a0, fullColumn
syscall
j     inputAndTurn
```

Print Updated Gameboard

printAndLoop:

```
jal    printBoard    # Call the function to print the updated game board.
j      inputAndTurn   # Restart the turn.
```

PART 3 - WIN/DRAW CONDITION LOGIC

VERTICAL WIN CHECK

Check Board

VerticalBoardCheck:

```
li     $t0, 0        # Initialize column counter ($t0) to 0.
lw     $t1, COLS      # Load the total number of columns
la     $t2, gameBoard # Load the base address of the game board into $t2.
```

checkNextColumn:

```
beq    $t0, $t1, verticalCheckDone
li     $t3, 7         # Start from the bottom row (row index 7).
li     $t4, 0         # Initialize Player 1's consecutive counter ($t4) to 0.
li     $t5, 0         # Initialize Player 2's consecutive counter ($t5) to 0.
```

checkNextRow:

```
mul    $t6, $t3, 8    # Calculate row offset: row index * 8 (8 columns per row).
add    $t6, $t6, $t0   # Add column offset to calculate cell index.
add    $t6, $t6, $t2   # Add base address of the game board to get cell's absolute
address.
lb     $t7, 0($t6)     # Load the value at the current cell into $t7.
```

Check for Player 1

```
li     $t8, '1'       # Load Player 1's piece ('1') into $t8.
beq    $t7, $t8, incrementP1
li     $t4, 0         # Otherwise, reset Player 1's consecutive counter to 0.
j      checkP2        # Jump to check Player 2's pieces.
```

incrementP1:

```
addi   $t4, $t4, 1    # Increment Player 1's consecutive counter.
beq    $t4, 4, playerOneWins
```

checkP2:

Check for Player 2

```

li    $t8, '2'          # Load Player 2's piece ('2') into $t8.
beq   $t7, $t8, incrementP2
li    $t5, 0            # Otherwise, reset Player 2's consecutive counter to 0.
j     nextRow

```

incrementP2:

```

addi   $t5, $t5, 1      # Increment Player 2's consecutive counter.
beq    $t5, 4, playerTwoWins

```

nextRow:

```

addi   $t3, $t3, -1     # Move to the row above by decrementing row index ($t3).
bgez   $t3, checkNextRow # If the row index is >= 0, continue checking rows in this
column.
addi   $t0, $t0, 1      # Move to the next column by incrementing column counter
($t0).
j      checkNextColumn

```

#Storing \$ra to the stack

verticalCheckDone:

```

addi   $sp, $sp, -4
sw     $ra, 0($sp)

```

CHILD (VERTICAL) TO GRANDCHILD (HORIZONTAL)

```

jal    HorizontalBoardCheck # Call the next function to check horizontal connections.

```

```

lw     $ra, 0($sp)
addi   $sp, $sp, 4
jr     $ra

```

#HORIZONTAL WIN CHECK

HorizontalBoardCheck:

```

li     $t0, 7           # Initialize row counter ($t0) to start from the bottom row (index 7).
lw     $t1, ROWS         # Load the total number of rows into $t1.
la     $t2, gameBoard    # Load the base address of the game board into $t2.

```

checkNextRowHorizontal:

```

bltz   $t0, horizontalCheckDone
li     $t3, 0            # Initialize column counter ($t3) to 0.
li     $t4, 0            # Reset Player 1's consecutive counter ($t4) to 0.

```

```
li    $t5, 0          # Reset Player 2's consecutive counter ($t5) to 0.
```

```
checkNextColumnHorizontal:
```

```
lw    $t6, COLS      # Load the total number of columns into $t6.
beq    $t3, $t6, nextRowHorizontal
mul    $t7, $t0, 8     # Calculate the row offset: row index * 8 (8 columns per row).
add    $t7, $t7, $t3   # Add column offset to calculate cell index.
add    $t7, $t7, $t2   # Add base address of the game board to get the absolute
address of the cell.
lb     $t8, 0($t7)     # Load the value at the current cell into $t8.
```

```
# Check for Player 1
```

```
li    $t9, '1'        # Load Player 1's piece ('1') into $t9.
beq    $t8, $t9, incrementHorizontalP1
li    $t4, 0           # Otherwise, reset Player 1's consecutive counter to 0.
j      checkHorizontalP2
```

```
incrementHorizontalP1:
```

```
addi   $t4, $t4, 1     # Increment Player 1's consecutive counter.
beq     $t4, 4, playerOneWins
j      nextColumnHorizontal # Otherwise, move to the next column.
```

```
checkHorizontalP2:
```

```
# Check for Player 2
li    $t9, '2'        # Load Player 2's piece ('2') into $t9.
beq    $t8, $t9, incrementHorizontalP2
li    $t5, 0           # Otherwise, reset Player 2's consecutive counter to 0.
j      nextColumnHorizontal # Jump to the next column.
```

```
incrementHorizontalP2:
```

```
addi   $t5, $t5, 1     # Increment Player 2's consecutive counter.
beq     $t5, 4, playerTwoWins
```

```
nextColumnHorizontal:
```

```
addi   $t3, $t3, 1     # Move to the next column by incrementing the column counter ($t3).
j      checkNextColumnHorizontal # Repeat the column-checking process.
```

```
nextRowHorizontal:
```

```
addi   $t0, $t0, -1    # Move to the row above by decrementing the row index ($t0).
```

```
j    checkNextRowHorizontal # Repeat the row-checking process.
```

```
#Storing $ra to the stack
```

```
horizontalCheckDone:
```

```
    addi    $sp, $sp, -4
```

```
    sw      $ra, 0($sp)
```

```
jal    DiagonalCheckForward # Nested call to DiagonalCheckForward
```

```
lw      $ra, 0($sp)
```

```
addi    $sp, $sp, 4
```

```
jr      $ra
```

```
# DIAGONAL WIN CHECK (Forward)
```

```
DiagonalCheckForward:
```

```
    li      $t0, 7          # Start from the bottom row (index 7).
```

```
forwardRowLoop:
```

```
    bltz    $t0, forwardDone
```

```
    li      $t1, 0
```

```
forwardColLoop:
```

```
    lw      $t2, COLS       # Load the number of columns.
```

```
    sub     $t2, $t2, 4
```

```
    bge     $t1, $t2, nextForwardRow
```

```
    li      $t3, 0          # Reset Player 1's piece counter.
```

```
    li      $t4, 0          # Reset Player 2's piece counter.
```

```
    move    $t5, $t0        # Set the starting row index for diagonal traversal.
```

```
    move    $t6, $t1        # Set the starting column index for diagonal traversal.
```

```
forwardDiagonalLoop:
```

```
    mul     $t7, $t5, 8      # Calculate the row's base offset: row_index * 8 columns.
```

```
    add     $t7, $t7, $t6
```

```
    la      $t8, gameBoard
```

```
    add     $t7, $t7, $t8    # Compute the absolute memory address of the current cell.
```

```
    lb      $t9, 0($t7)
```

```
# Check for Player 1
```

```
    li      $s0, '1'        # Load '1' to represent Player 1's pieces.
```

```

beq  $t9, $s0, incrementForwardP1
li   $t3, 0          # Otherwise, reset Player 1's counter.
j    checkForwardP2

```

incrementForwardP1:

```

addi  $t3, $t3, 1      # Increment Player 1's consecutive piece counter.
beq   $t3, 4, playerOneWins
j     nextForwardDiagonal # Continue checking the next diagonal cell.

```

checkForwardP2:

```

# Check for Player 2
li   $s0, '2'          # Load '2' to represent Player 2's pieces.
beq  $t9, $s0, incrementForwardP2
li   $t4, 0            # Otherwise, reset Player 2's counter.
j    nextForwardDiagonal

```

incrementForwardP2:

```

addi  $t4, $t4, 1      # Increment Player 2's consecutive piece counter.
beq   $t4, 4, playerTwoWins

```

nextForwardDiagonal:

```

addi  $t5, $t5, -1     # Move diagonally up (row index decreases).
addi  $t6, $t6, 1      # Move diagonally to the right (column index increases).
bgez  $t5, forwardDiagonalLoop
blt   $t6, $t2, forwardDiagonalLoop

```

```

addi  $t1, $t1, 1
j     forwardColLoop

```

nextForwardRow:

```

addi  $t0, $t0, -1     # Move to the next starting row (one row up).
j     forwardRowLoop

```

#Storing \$ra to the stack

forwardDone:

```

addi  $sp, $sp, -4
sw    $ra, 0($sp)

```

```

jal   DiagonalCheckBackward # Nested call to Diagonal CheckBackward

```

```
lw    $ra, 0($sp)
addi  $sp, $sp, 4
jr    $ra
```

DIAGONAL WIN CHECK (BACKWARD)

DiagonalCheckBackward:

```
li    $t0, 7          # Start from the bottom row
backwardRowLoop:
    bltz  $t0, backwardDone
    lw    $t1, COLS
    subi  $t1, $t1, 1
```

backwardColLoop:

```
    bltz  $t1, nextBackwardRow # If all valid columns are checked, move to next row
```

```
li    $t3, 0
li    $t4, 0
move  $t5, $t0
move  $t6, $t1
```

backwardDiagonalLoop:

```
mul    $t7, $t5, 8      # Row offset = row_index * 8
add    $t7, $t7, $t6
la     $t8, gameBoard
add    $t7, $t7, $t8
lb     $t9, 0($t7)
```

Check for Player 1

```
li    $s0, '1'
beq    $t9, $s0, incrementBackwardP1
li    $t3, 0          # Reset Player 1 counter
j      checkBackwardP2
```

incrementBackwardP1:

```
addi  $t3, $t3, 1
beq    $t3, 4, playerOneWins
j      nextBackwardDiagonal
```

checkBackwardP2:

```
# Check for Player 2
li    $s0, '2'
beq   $t9, $s0, incrementBackwardP2
li    $t4, 0      # Reset Player 2 counter
j     nextBackwardDiagonal
```

incrementBackwardP2:

```
addi   $t4, $t4, 1
beq    $t4, 4, playerTwoWins
```

nextBackwardDiagonal:

```
addi   $t5, $t5, -1    # Move to the next diagonal cell (up a row)
addi   $t6, $t6, -1    # Move to the next diagonal cell (left a column)
bgez   $t5, backwardDiagonalLoop
bgez   $t6, backwardDiagonalLoop
```

```
addi   $t1, $t1, -1    # Move to the next column in the starting row
j      backwardColLoop
```

nextBackwardRow:

```
addi   $t0, $t0, -1    # Move to the next starting row (up)
j      backwardRowLoop
```

#Storing \$ra to the stack

backwardDone:

```
addi   $sp, $sp, -4
sw     $ra, 0($sp)
```

```
jal    checkDraw # Nested call to checkDraw
```

```
lw     $ra, 0($sp)
addi   $sp, $sp, 4
jr     $ra
```

#DRAW CONDITION

checkDraw:

```
la    $t0, gameBoard
lw    $t1, ROWS
lw    $t2, COLS
mul   $t1, $t1, $t2
li    $t3, 0
```

drawCheckLoop:

```
lb    $t4, 0($t0)
li    $t5, ''
beq   $t4, $t5, notDraw
addi  $t3, $t3, 1
addi  $t0, $t0, 1
blt   $t3, $t1, drawCheckLoop
```

Draw Message String

```
li    $v0, 4
la    $a0, drawCondition
syscall
j     gameEnd
```

notDraw:

```
jr    $ra
```

WIN CONDITION MESSAGES

playerOneWins: # Win String Player 1 Print

```
li    $v0, 4
la    $a0, winCondition1
syscall
li    $t8, 1
j     gameEnd
```

playerTwoWins: # Win String Player 2 Print

```
li    $v0, 4
la    $a0, winCondition2
syscall
li    $t8, 2
j     gameEnd
```

End of Game with Sound

gameEnd:

```
    beq    $t8, 1, playPlayerOneSound # If Player 1 won, play their sound
    beq    $t8, 2, playPlayerTwoSound # If Player 2 won, play their sound
    j      exitGame
```

playPlayerOneSound:

```
    # MIDI sound for Player 1
    li     $v0, 31          # Sound Syscall
    la     $a0, 67          # Pitch
    la     $a1, 100         # Duration
    la     $a2, 56          # Instrument Type
    la     $a3, 127         # Volume
    syscall
    j      exitGame
```

playPlayerTwoSound:

```
    # Player 2 Win Sound
    li     $v0, 31          # Sound Syscall
    la     $a0, 60          # Pitch
    la     $a1, 100         # Duration
    la     $a2, 19          # Instrument Type
    la     $a3, 127         # Volume
    syscall
    j      exitGame
```

#Exit Syscall to End Game/Program

exitGame:

```
    li     $v0, 10
    syscall
```